# HiTrace Solutions — LoanNBFC (Gidfine)

Frontend Architecture & Migration Diagnosis Report | Date: October 17, 2026 | Branch: development

## 1. Executive Summary & Direction Conflict Analysis

During Phase 1 inspection of the repository on branch **development**, an architectural conflict was identified between the requested migration objective and the repository's active migration documentation:

| | |
|---|---|
| **Requested Direction:** Vue.js → React | **Documented in Repo:** React 19 → Vue 3 (VUE_MIGRATION_PLAN.md, Oct 12 2026) |
| Client objective is to maintain React as the primary production frontend with Convex data platform integration. | Work was previously underway to rewrite React components into Vue 3 (Composition API, Pinia, @vueuse/motion). |

**Key Finding:** The original React 19 codebase remains **100% intact** inside frontend/src/. Existing .tsx components, the Zustand state store (store/useLoanStore.ts), and React hooks were not deleted. However, Vite configuration (vite.config.ts) and index.html were pointed to Vue entrypoints (main.ts / App.vue). Moving in the requested Vue → React direction is achievable with zero loss of functionality by switching the build entry back to React and validating all React modules.

## 2. Status of Convex Integration

An exhaustive scan of the repository confirmed that **Convex is not currently configured or installed**:
• **Configuration files:** No convex.json or convex/ directory exists.
• **Dependencies:** Zero Convex packages in frontend/package.json or Python requirements.txt.
• **Current backend:** The production backend is a **FastAPI** async service connecting to **MongoDB Atlas (Motor)** with LangGraph deterministic agent pipelines (agents/) and WebSocket session management. All API calls in src/lib/api.ts target FastAPI endpoints.
• **Recommendation:** Convex should be introduced strictly for MVP reactive state/data synchronization while preserving the FastAPI financial rules engine, underwriting, OCR forensics, and ReportLab PDF services.

## 3. Frontend Architecture Inventory & Component Status

| Layer / Area | React Stack (Preserved) | Vue Stack (Duplicate/WIP) | Migration Action |
|---|---|---|---|
| Framework & Build | React 19.2.4 + Vite 5.4 | Vue 3.5.43 + Vite 5.4 | Point Vite plugin to React |
| Application Entry | src/main.tsx + src/App.tsx | src/main.ts + src/App.vue | Restore index.html script tag |
| State Management | Zustand 5.0 (useLoanStore.ts) | Pinia 3.0 (loanStore.ts) | Keep Zustand as primary store |
| Styling System | Tailwind CSS v3 + shadcn/ui | Tailwind CSS v3 (same config) | Shared CSS tokens reused |
| Animations & UI | Framer Motion 12 + lucide-react | @vueuse/motion + lucide-vue | Maintain Framer Motion |
| Charts & Analytics | Recharts 3.8 | Chart.js + vue-chartjs | Preserve Recharts visuals |
| API Client Layer | src/lib/api.ts (REST + WS) | Direct port (same endpoints) | 100% reusable as-is |

## 4. Critical Business Features Verified for React Target

1. **Multi-Step Onboarding:** Interactive borrower journey with credit assessment and KYC collection.
2. **AI Advisor Chat & WebSocket:** Real-time streaming assistant with agent thinking indicators and decision traces.
3. **Underwriting & Explainability:** SHAP explainability charts, loan offer cards, and multi-lender comparison table.
4. **What-If Financial Simulator:** Interactive slider recalculating loan amount, tenures, EMIs, and affordability.
5. **Sanction & Disbursal:** PDF sanction letter viewer, digital signature acceptance, and repayment tracking.

**6. Admin Operations:** Lender performance management, conversion funnel statistics, and system KPI dashboards.

## 5. Recommended Action Plan & Next Verification Steps

• **Step 1:** Install Node dependencies (npm install in frontend/) to restore build tooling.
• **Step 2:** Switch frontend/vite.config.ts to use @vitejs/plugin-react and update index.html to /src/main.tsx.
• **Step 3:** Run TypeScript type-check (tsc -b) and production bundle build (vite build) on React tree.
• **Step 4:** Evaluate Convex schema specification and determine integration boundary without breaking existing FastAPI backend.